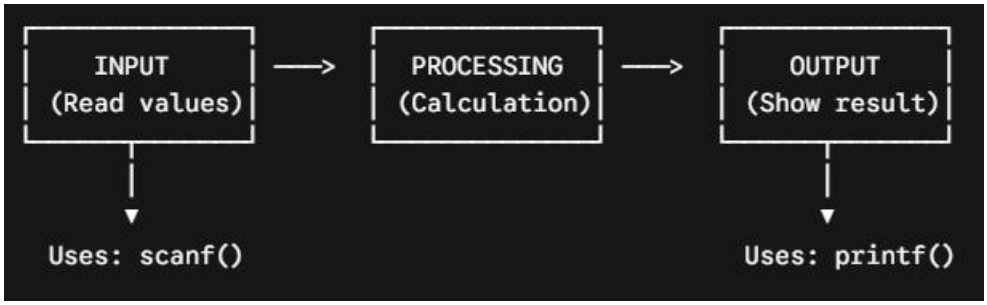
The Core Paradigm: Input --> Processing --> Output

Every computer program operates on a simple, foundational pipeline:



In C, these operations are handled by standard library functions contained within the **stdio.h** header file.

std = Standard

i = Input

o = Output

.h = Header File (contains pre-made function declarations)

To use these functions, you must always include this line at the very top of your program:

#include <stdio.h>

1. Output in C: The printf() Function

printf stands for **print formatted**. It sends data to the standard output device (typically your monitor screen).

Syntax

printf("Your message here");

printf("Format Specifiers", variable1, variable2);

◇ Example 1: Printing a Simple Message

```
#include <stdio.h>
int main() {
```

```
  printf("Welcome to C Programming");
  return 0;
}
```

Output:

Welcome to C Programming

💡 **What does return 0; mean?** > For now, think of it as a signaling flag. It tells your Operating System (OS) that the program executed successfully from start to finish without crashing.

Later, when we study **functions**, we'll understand the deeper meaning of the return statement.

◇ Example 2: The Need for Escape Sequences

```
#include <stdio.h>
int main() {
  printf("Hello");
  printf("World");
  return 0;
}
```

Output:

HelloWorld

Notice how the words are smashed together? printf() does **not** automatically insert spaces or new lines. To clean this up, we use the **New Line Character (\n)**.

The New Line Character (\n)

\n is an escape sequence that forces the screen cursor to jump down to the next line.

```
printf("Hello\n");printf("World");
```

Output:

Hello
World

🎯 2. What is a Format Specifier?

A format specifier acts as a placeholder inside a string. It tells printf() or scanf() exactly **what type of data** it needs to handle. Format specifiers always start with a percent sign (%).

Quick Reference Table

Data Type	Format Specifier	Typical Data Guarded
int	%d or %i	Whole numbers (Signed Integer)
unsigned int	%u	Positive-only whole numbers
long int	%ld	Very large integers
char	%c	Single text characters / symbols
float	%f	Standard decimal numbers
double	%lf	Long (double precision) decimals
string	%s	Sequences of multiple characters

◇ Example: Printing a Variable Value

```
#include <stdio.h>
int main() {
   int age = 20;
   printf("I am %d years old.", age); // %d is replaced by the value of age
   return 0;
}
```

Output:

I am 20 years old.

⌨ 3. Input in C: The scanf() Function

scanf stands for **scan formatted**. It pauses program execution and waits for the user to type data on the keyboard.

Syntax

```
scanf("format_specifier", &variable_name);
```

◇ Example 1: Reading a Single Value

```
#include <stdio.h>
int main() {
  int age;

  printf("Enter age: ");
  scanf("%d", &age); // Waits for user input

  printf("Your age is %d", age);
  return 0;
}
```

⚓ Why is the Address-of Operator (&) Crucial?

When you declare a variable like int age;, your computer assigns a specific, numbered physical slot in its RAM to hold that value.

age refers to the *value* stored in the box.

&age refers to the *location address* of the box in memory.

When you use scanf(), the program needs to know exactly **where** in the computer memory map to write the incoming data.

scanf("%d", &age); --> "Find an integer from the keyboard, track down the physical address of the variable 'age', and drop the value right into that memory slot."

👥 4. Handling Multiple Inputs and Outputs

You can read or write multiple values within a single line of code by matching the format specifiers sequentially from left to right.

◇ Code Example

```
#include <stdio.h>
int main() {
  int a;
  int b; // Corrected initialization syntax error from original notes

  printf("Enter the value of a and b: \n");

  // Accepts two integers separated by a space or enter key
  scanf("%d%d", &a, &b);
```

```
    // Prints both values back out separated by a space
    printf("You entered: %d and %d\n", a, b);

    return 0;
}
```

Execution Breakdown

Execution Screen:

```
----------------------------------------
Enter the value of a and b:
10 20                  <-- (User types this and hits enter)
You entered: 10 and 20     <-- (Program execution output)
----------------------------------------
```

Inside scanf("%d%d", &a, &b);:

The first %d maps to &a ---> a gets 10

The second %d maps to &b ---> b gets 20